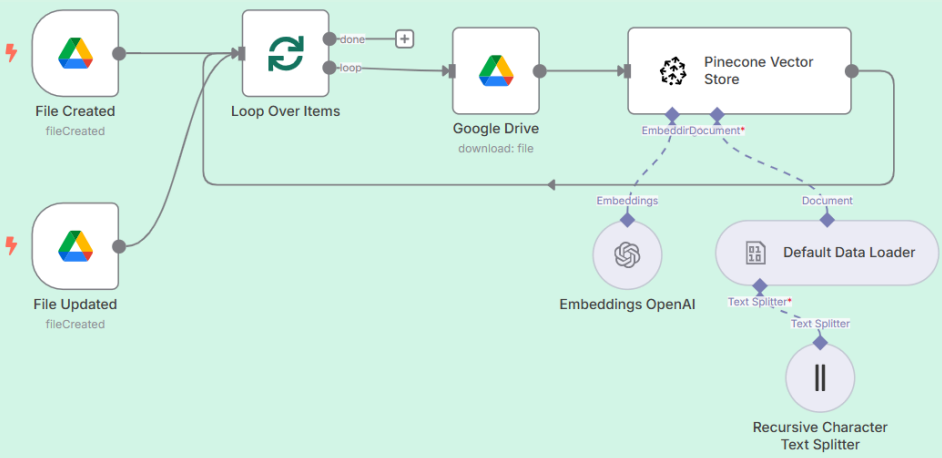
RAG Architectures in Practice

Start Building. Stop Theorizing.

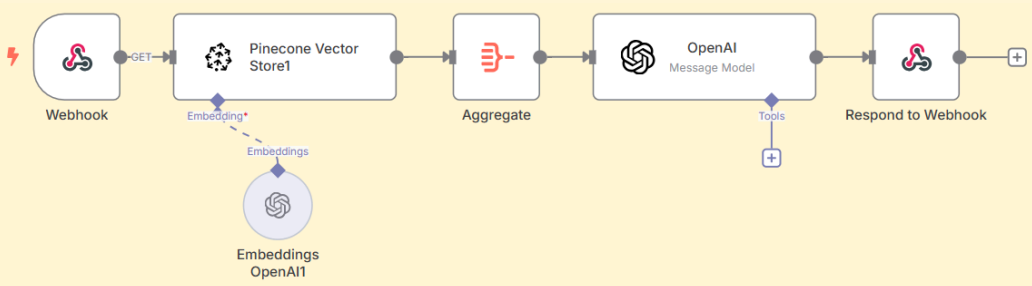
Prep: Add Google Drive files to Pinecone

Reacts when files are added or modified in my folder



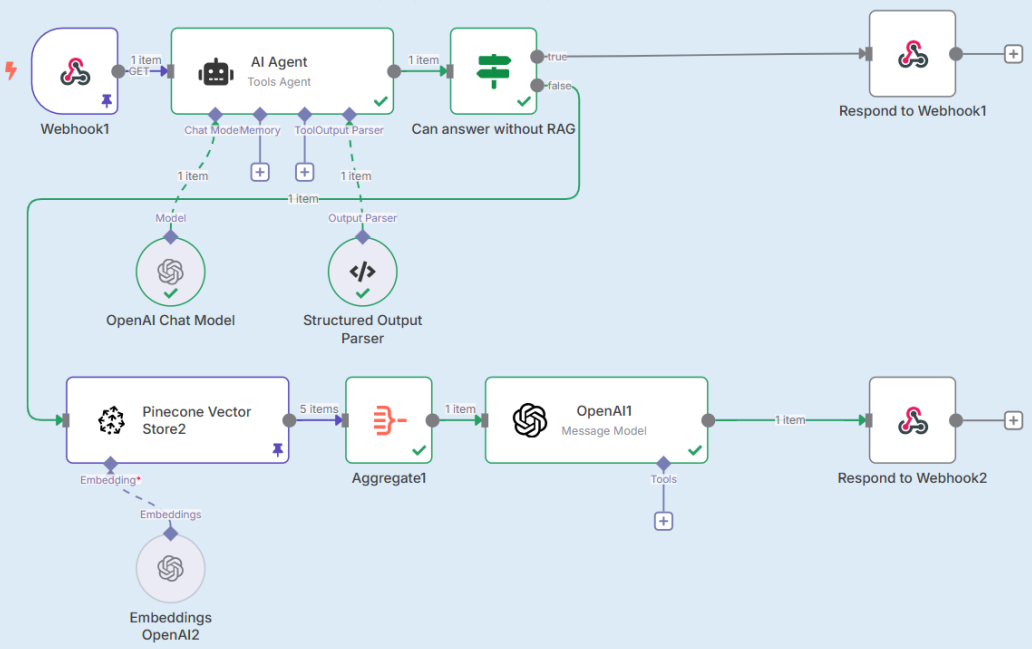
Vanilla RAG

Reuses user request as a RAG prompt



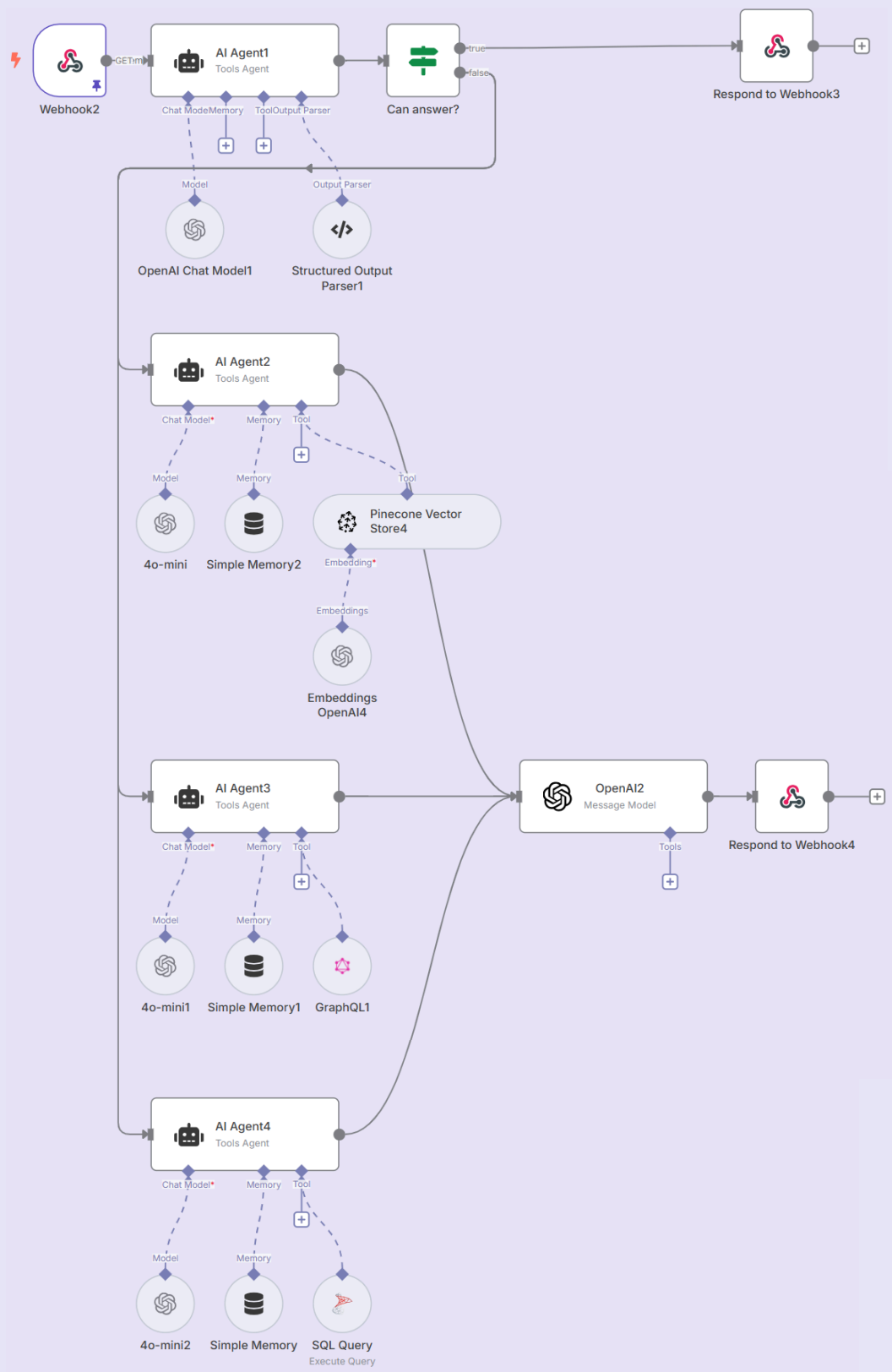
Adaptive RAG

Can answer without RAG, formulates a decidated RAG prompt



Hybrid RAG

Can query multiple data sources. Here, demonstrated for Pinecone vector database, GraphQL, and SQL

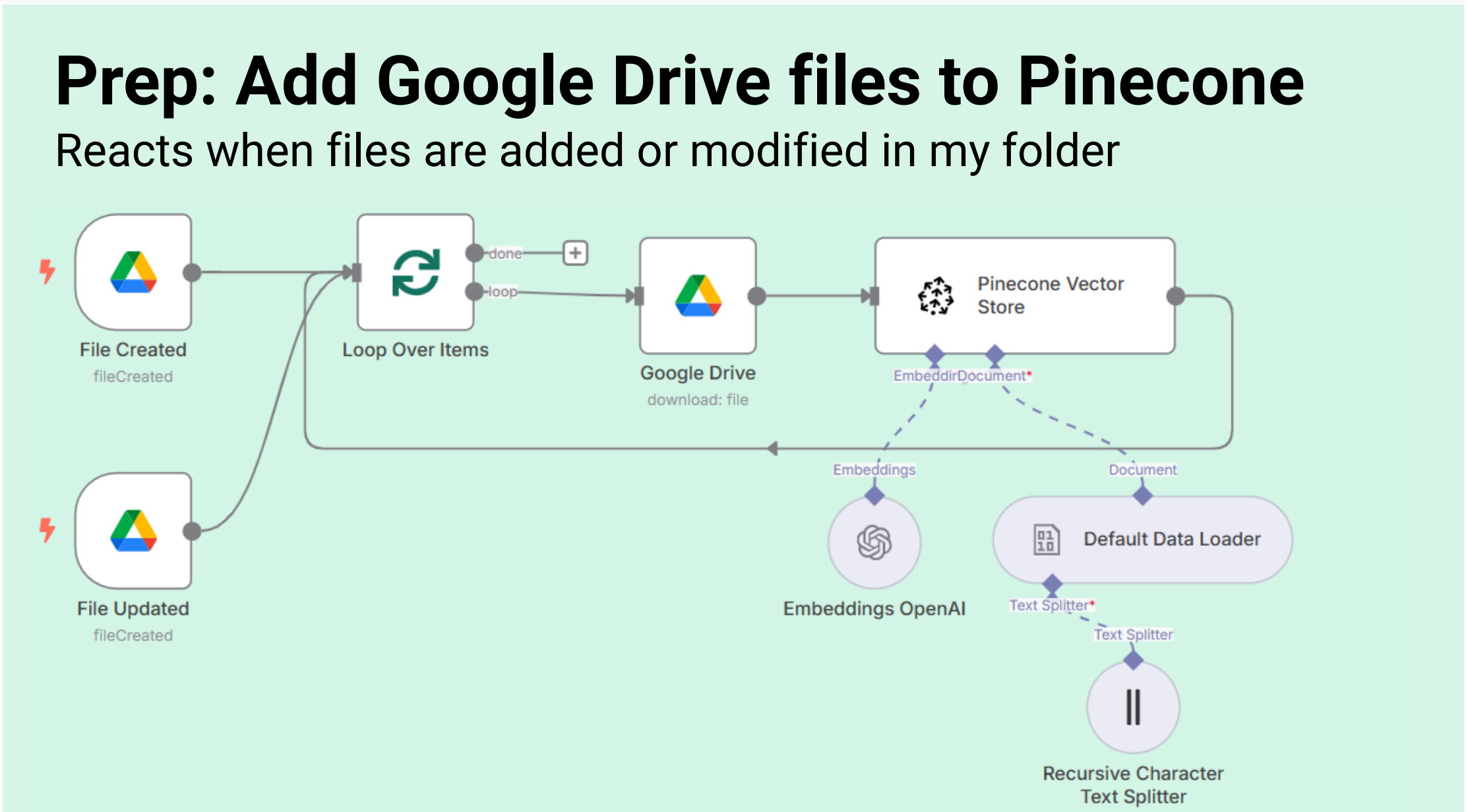


The Product Compass Newsletter

Let's start with generating embeddings for documents in my Google Drive folder. I will use free Pinecone.

We need to:

- Split documents into chunks
- Store them as multi-dimensional vectors

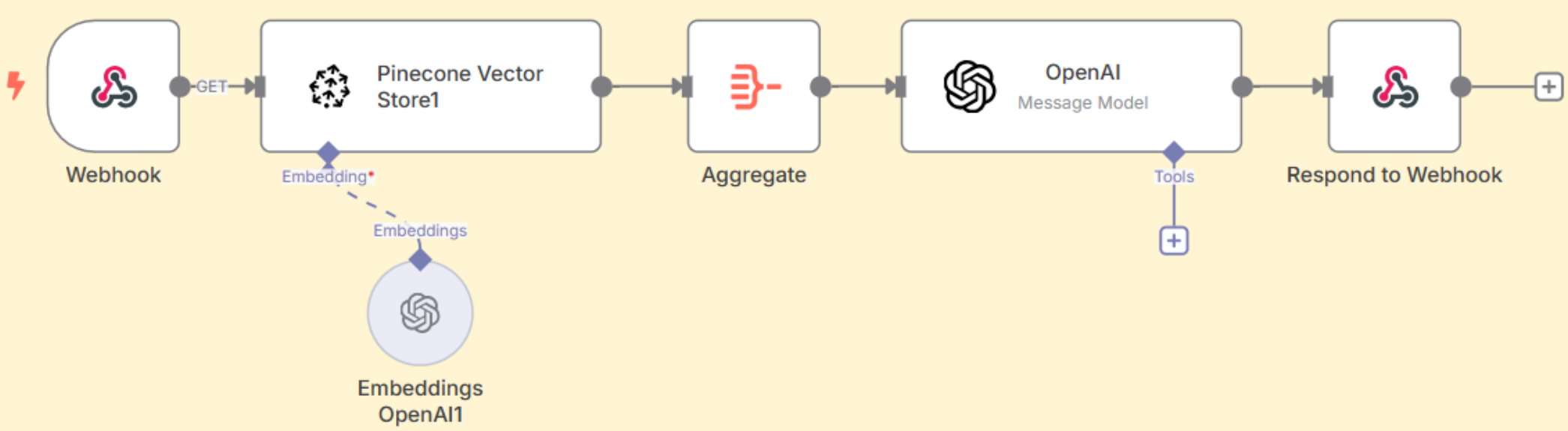


Now, when the user asks a question, we can convert it into a vector and retrieve document chunks with similar vectors from Pinecone.

Then, use those chunks to provide the answer:

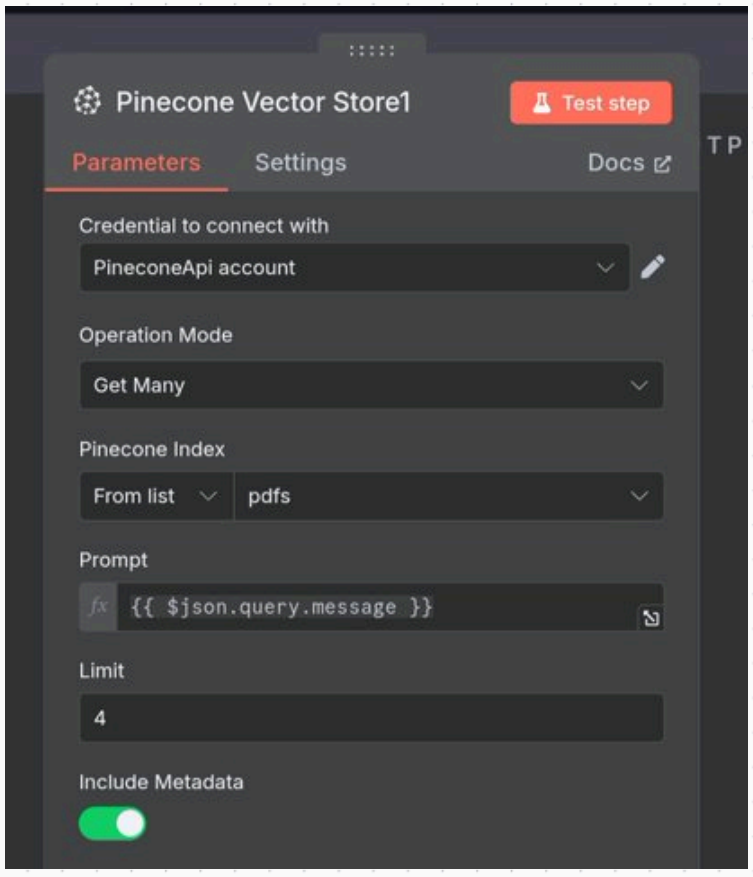
Vanilla RAG

Reuses user request as a RAG prompt



TLDR; You can play with the number of chunks or re-ranking them. The latter can often be done as part of the final prompt.

No need to complicate those techniques with fancy names to memorize.

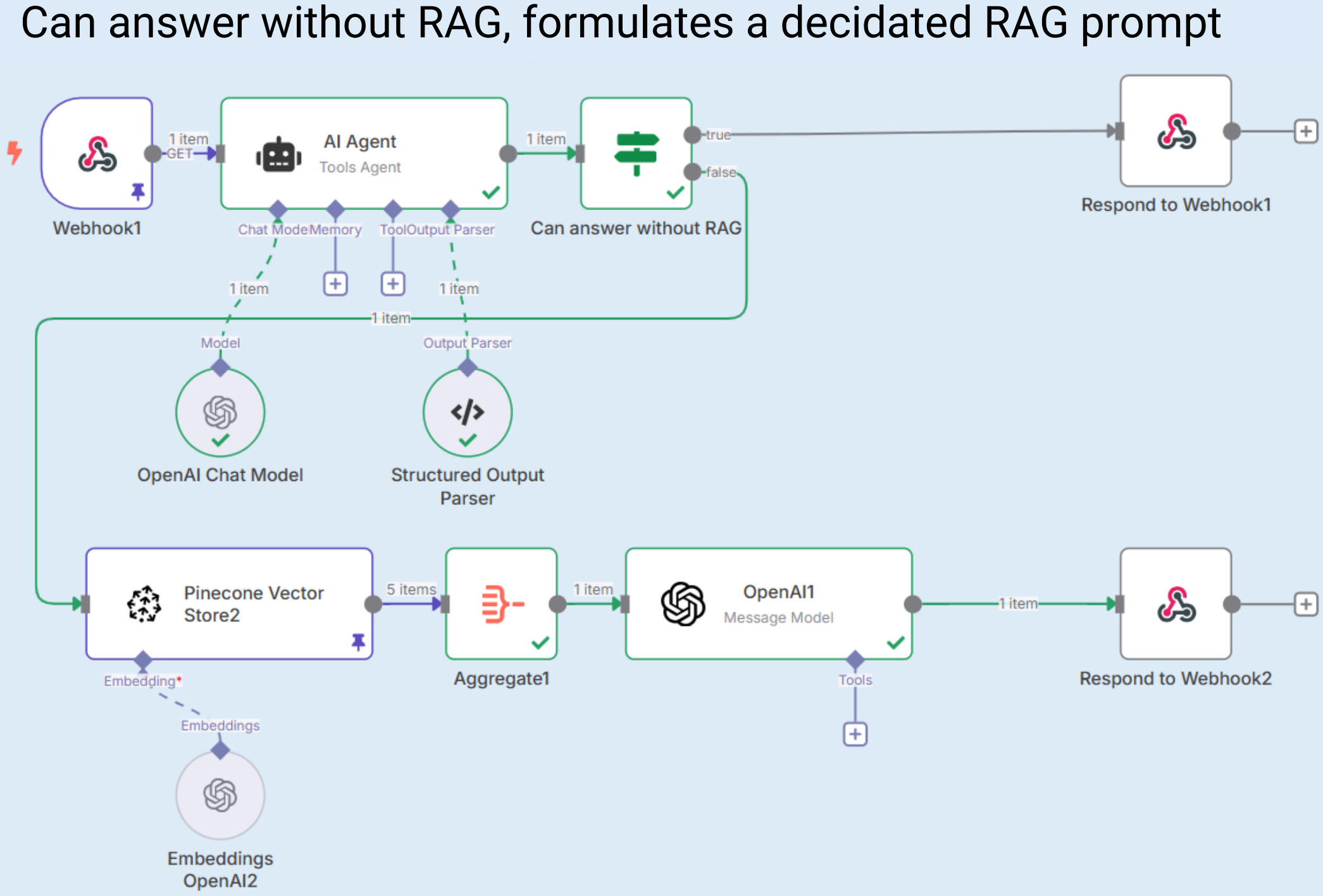


But the user might have provided many unnecessary details.

It’s more effective to remove the noise by creating a dedicated Pinecone prompt. If you need to prompt at all!

Adaptive RAG

Can answer without RAG, formulates a decidated RAG prompt



We can complicate things over time, e.g., add more complex retrieval strategies. But the core idea stays the same.

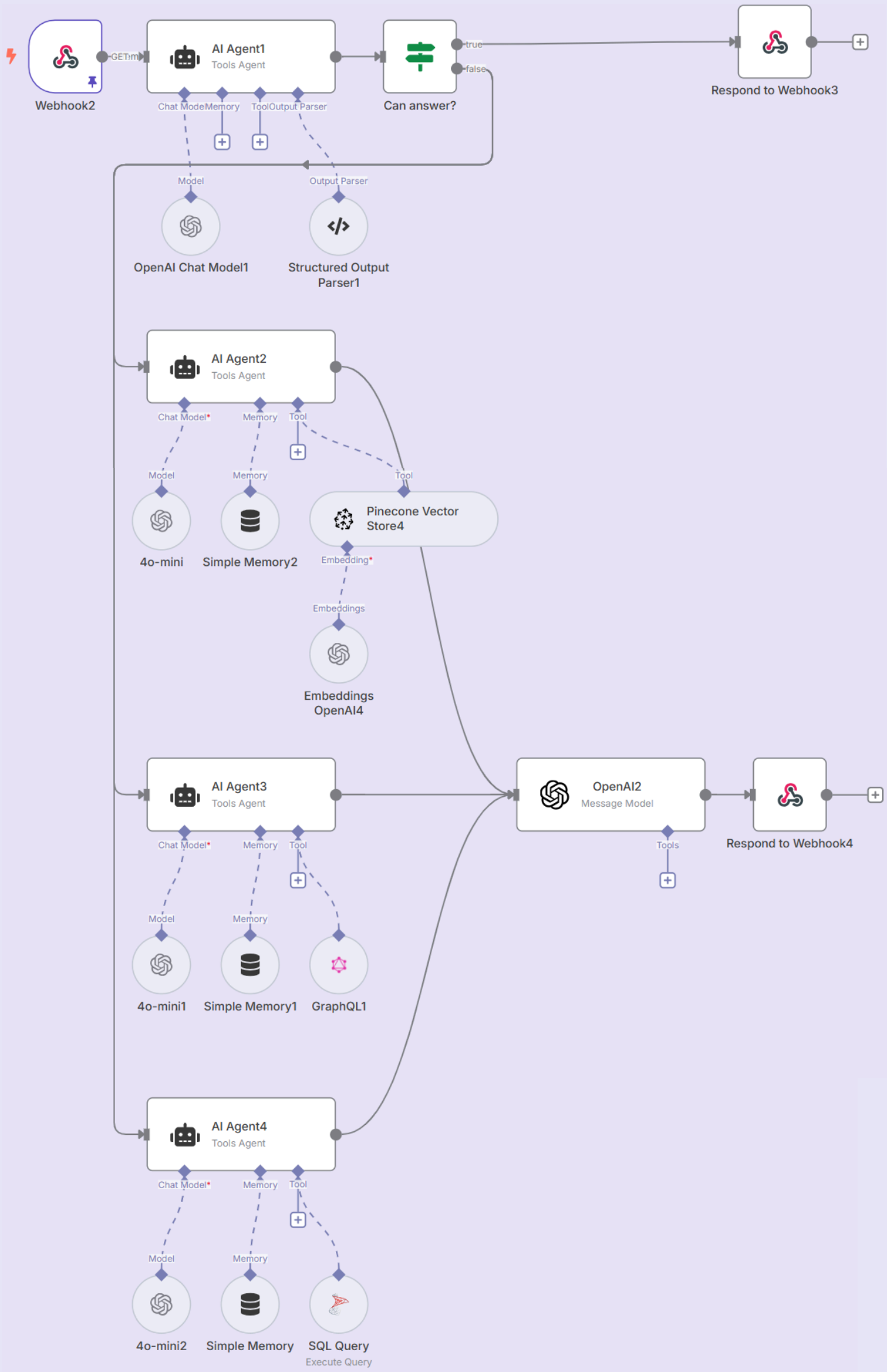
Your product might have multiple data sources: Pinecone, Graph DB, SQL.

Here, I look for information everywhere without filtering data sources.

Finally, I used a dedicated prompt to prepare the answer.

Hybrid RAG

Can query multiple data sources. Here, demonstrated for Pinecone vector database, GraphQL, and SQL



Hope that helps!

My poster as an n8n workflow template (json, Google Drive) is in the post!